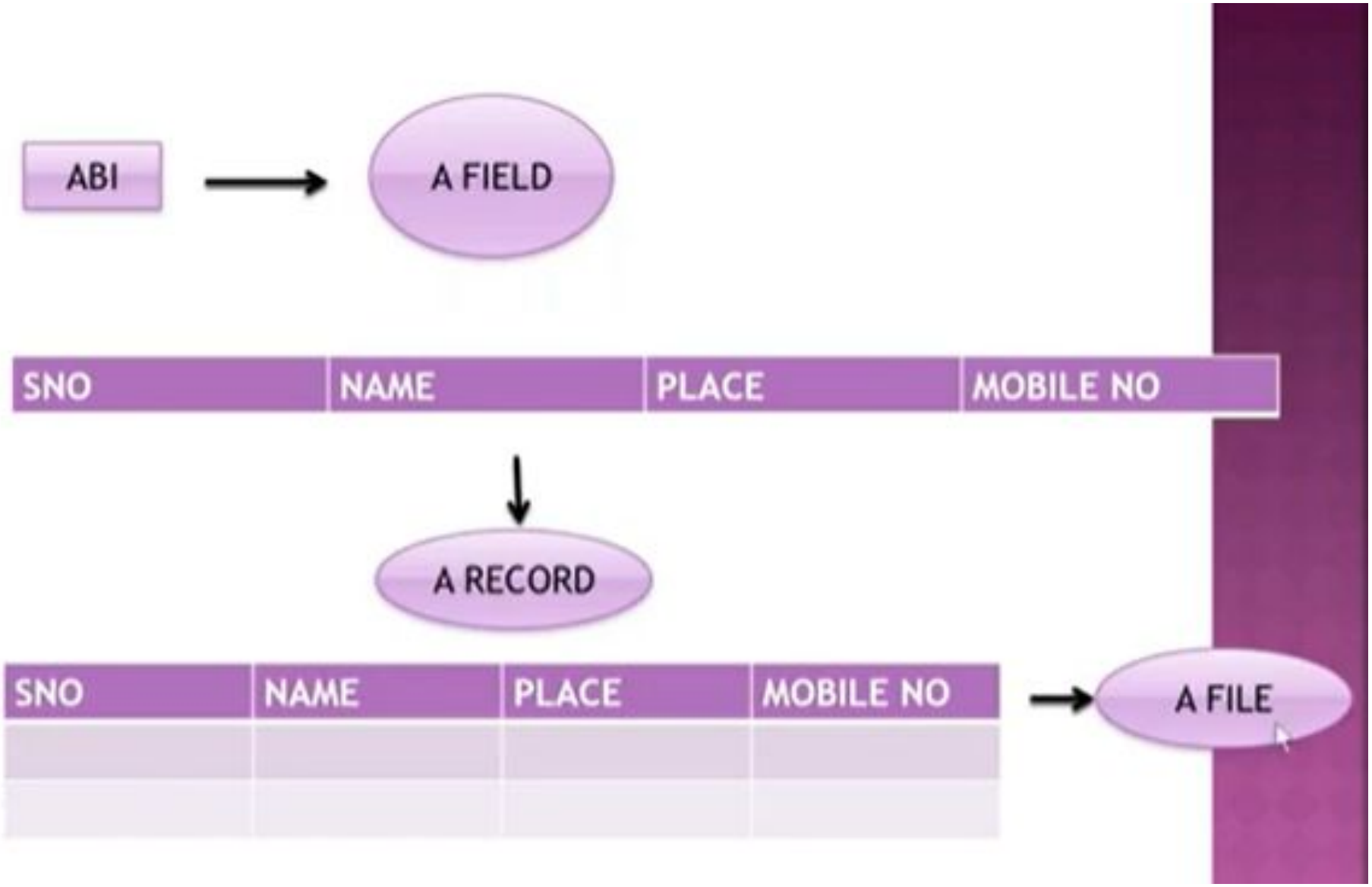
FILE ORGANIZATION

https://youtu.be/W9vvrKzzV60?si=EwbLUD-Ky0PC2Jrf

FILE ORGANIZATION

- The database is stored as a collection of *files*. Each file is a sequence of *records*. A record is a sequence of fields.
- **Logical Structure of a file**
  - **Field:** Smallest (fixed) indivisible logical unit of a file. A field holds a part of some data value.
  - **Record:** A set of logically related fields.

File Representation



Organization of Records in Files

- Classifications of records
  - - Fixed length record
  - - Variable length record
- Fixed length record approach:
  - Assume record size is fixed each file has records of one particular type only different files are used for different relations.
  - This case is easiest to implement

FIXED-LENGTH RECORDS

- Simple approach:
  - Store record *i* starting from byte $n * (i - 1)$, where *n* is the size of each record.
  - Record access is simple but records may cross blocks
    - **Modification: do not allow records to cross block boundaries**
- Deletion of record *i*: alternatives:
  - move records $i + 1, \ldots, n$ to $i, \ldots, n - 1$
  - move record *n* to *i*
  - do not move records, but link all free records on a *free list*

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

FIXED-LENGTH RECORDS -- DELETION

1. DELETING RECORD 3 AND COMPACTING

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

FIXED-LENGTH RECORDS -- DELETION

2. DELETING RECORD 3 AND MOVING LAST RECORD

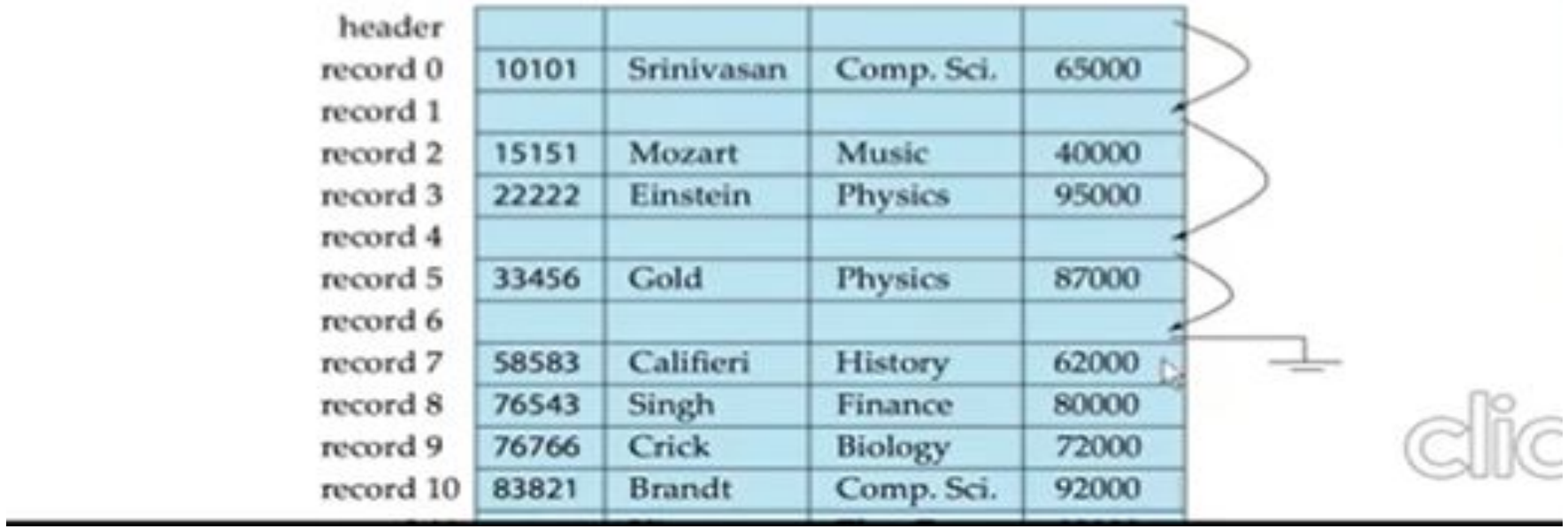
DELETING RECORD 3 AND MOVING LAST RECORD

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

FIXED-LENGTH RECORDS -- DELETION

3. FREE LISTS

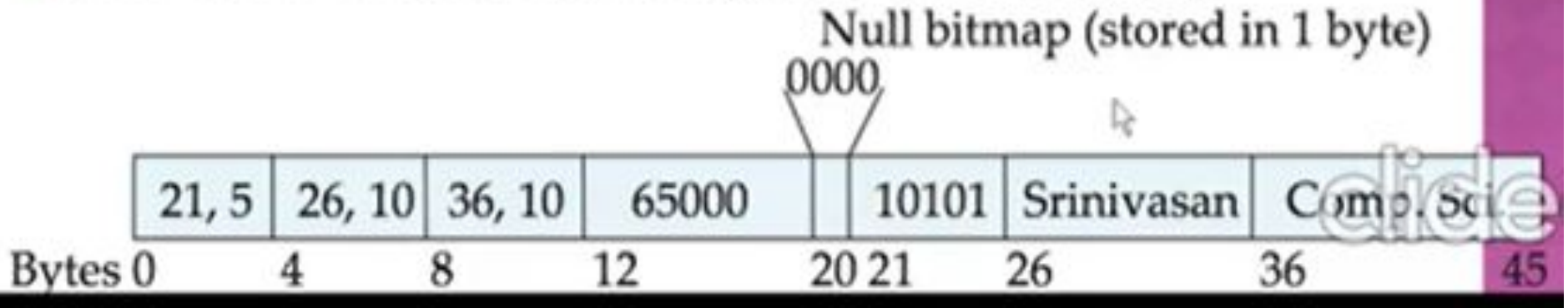
- Store the address of the first deleted record in the file header.
- Use this first record to store the address of the second deleted record, and so on
- Can think of these stored addresses as pointers since they "point" to the location of a record.
- More space efficient representation: reuse space for normal attributes of free records to store pointers. (No pointers stored in in-use records.)



header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

VARIABLE-LENGTH RECORDS

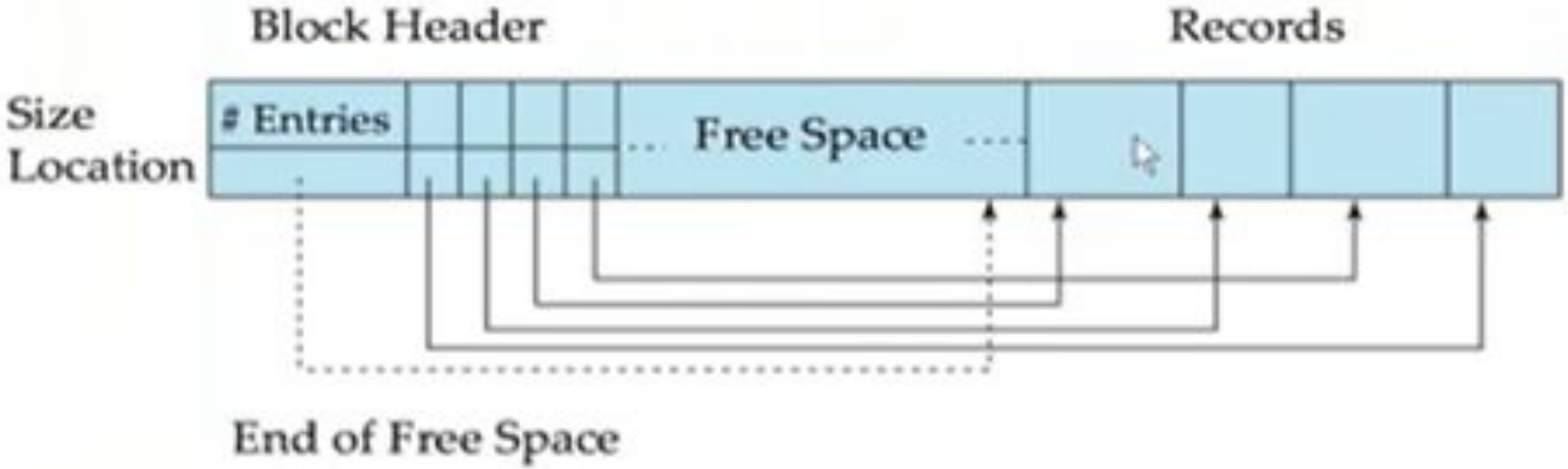
- Variable-length records arise in database systems in several ways:
  - Storage of multiple record types in a file.
  - Record types that allow variable lengths for one or more fields such as strings (**varchar**)
  - Record types that allow repeating fields (used in some older data models).
- Attributes are stored in order
- Variable length attributes represented by fixed size (offset, length), with actual data stored after all fixed length attributes
- Null values represented by null-value bitmap



The length of the attribute (field) and the Length of the record can be changed according to the data inserted

Can contain NULL values for some fields

VARIABLE-LENGTH RECORDS: SLOTTED PAGE STRUCTURE

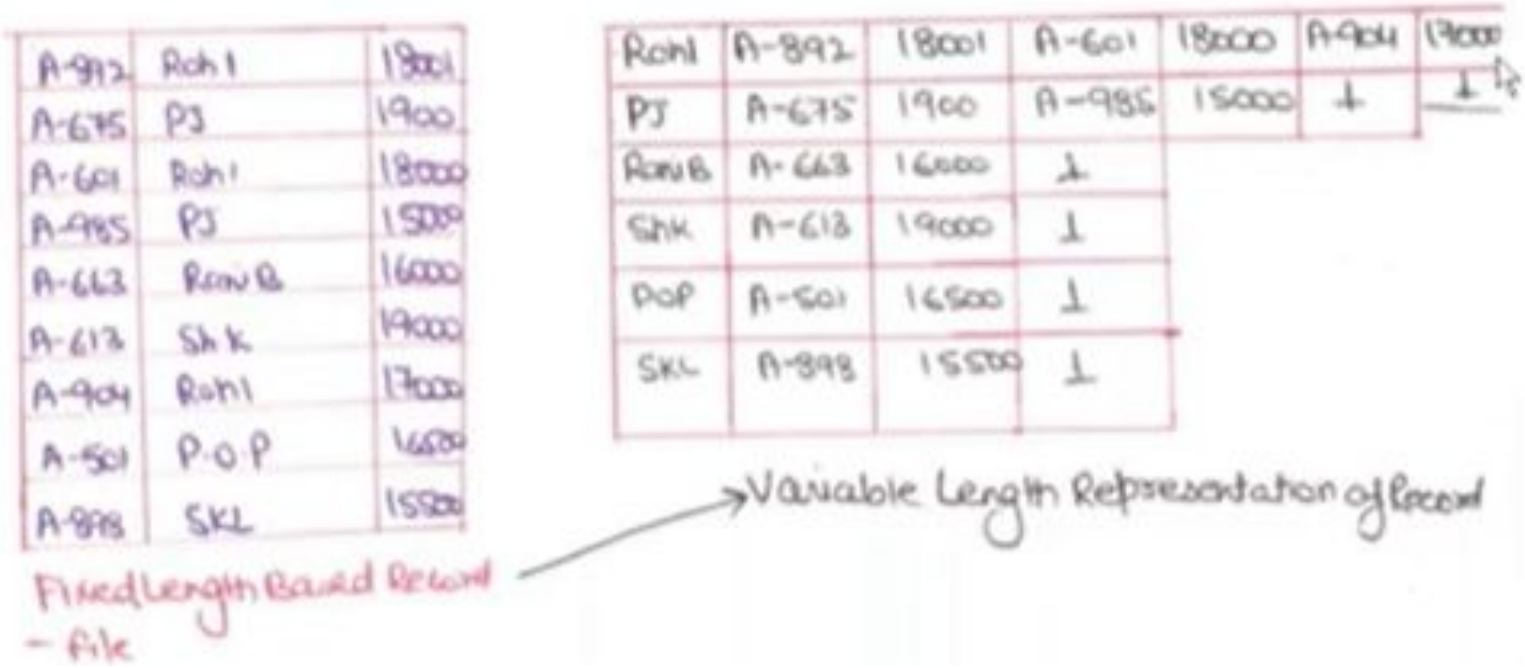


- **Slotted page** header contains:
  - number of record entries
  - end of free space in the block
  - location and size of each record
- Records can be moved around within a page to keep them contiguous with no empty space between them; entry in the header must be updated.
- Pointers should not point directly to record — instead they should point to the entry for the record in header.

clideo.

Difference between Fixed-length and Variable-length records

The differences between fixed length record and variable length is that the fixed length record should have values for all fields. The fields cannot have NULL values. Variable length records can have NULL values for fields.



FILE ORGANIZATION

FILE ORGANIZATION

- The **File** is a collection of records. Using the primary key, we can access the records.
- File organization is a logical relationship among various records. This method defines how file records are mapped onto disk blocks.
- File organization is used to describe the way in which the records are stored in terms of blocks, and the blocks are placed on the storage medium.
  - The first approach to map the database to the file is to use the several files and store only one fixed length record in any given file.
  - An alternative approach is to structure our files so that we can contain multiple lengths for records.
- Files of fixed length records are easier to implement than the files of variable length records.

FILE ORGANIZATION

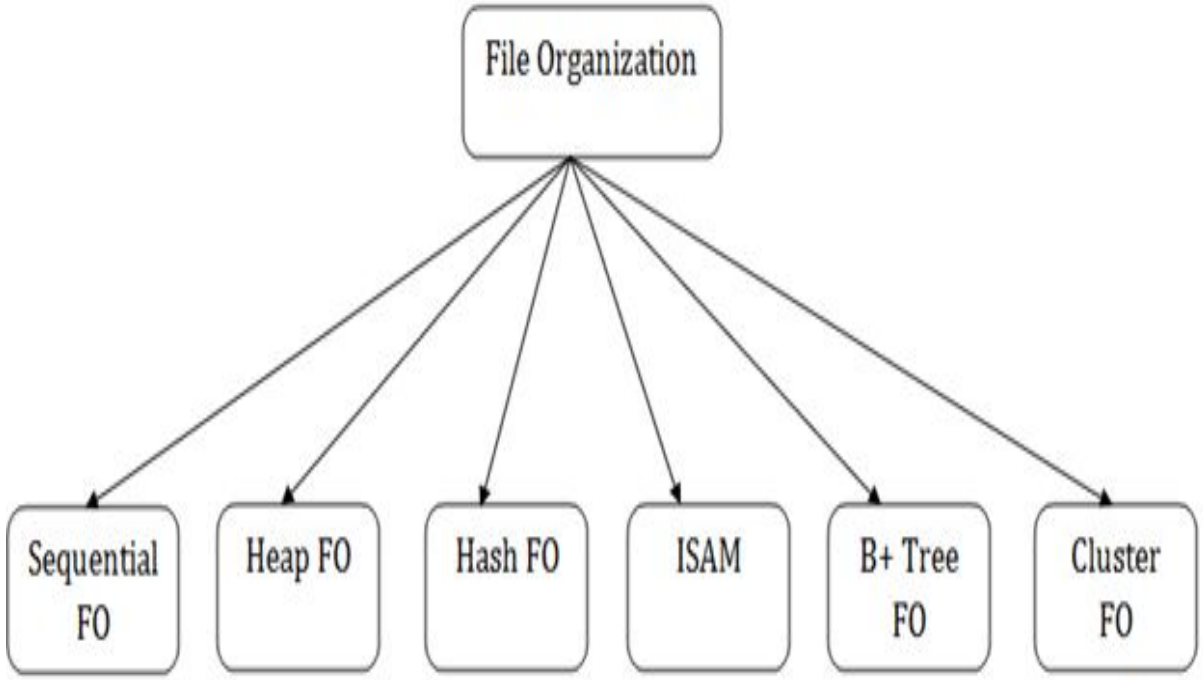
- Objective of file organization

  - It contains an optimal selection of records, i.e., records can be selected as fast as possible.
  - To perform insert, delete or update transaction on the records should be quick and easy.
  - The duplicate records cannot be induced as a result of insert, update or delete.
  - For the minimal cost of storage, records should be stored efficiently.

FILE ORGANIZATION

Types of file organization:

- File organization contains various methods.
- Each method has its own pros and cons on the basis of access or selection
- In the file organization, the programmer decides the best-suited file organization method according to his requirement.



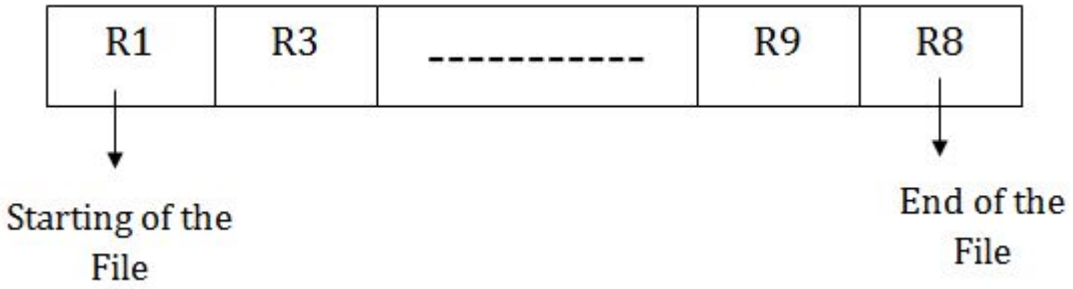
FILE ORGANIZATION

1. Sequential File Organization

- Easiest method for file organization. In this method, files are stored sequentially. This method can be implemented in two ways:

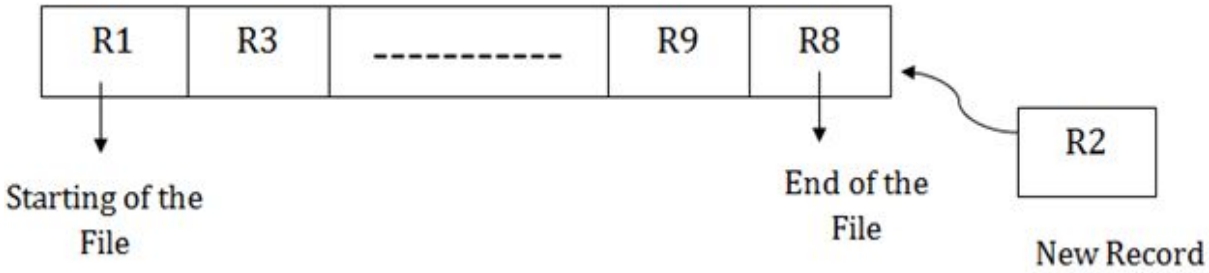
1. Pile File Method:

- It is a quite simple method. In this method, we store the record in a sequence, i.e., one after another.
- Records are nothing but a row in the table.
- Here, the record will be inserted in the order in which they are inserted into tables.
- In case of updating or deleting of any record, the record will be searched in the memory blocks. When it is found, then it will be marked for deleting, and the new record is inserted.



Insertion of the new record:

- Suppose we have the records R1, R3 and so on up to R9 and R8 in a sequence.
- Suppose we want to insert a new record R2 in the sequence, then it will be placed at the end of the file

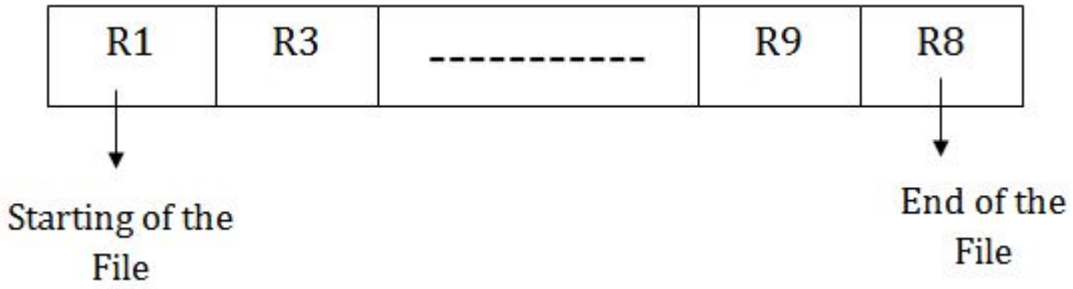


FILE ORGANIZATION

1. Sequential File Organization

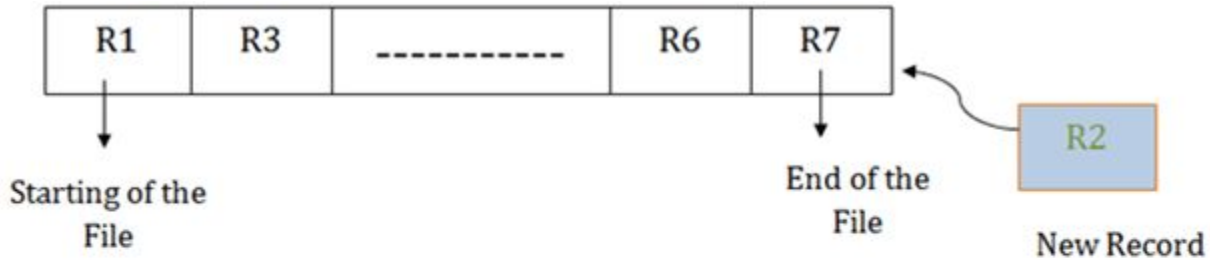
2. Sorted File Method:

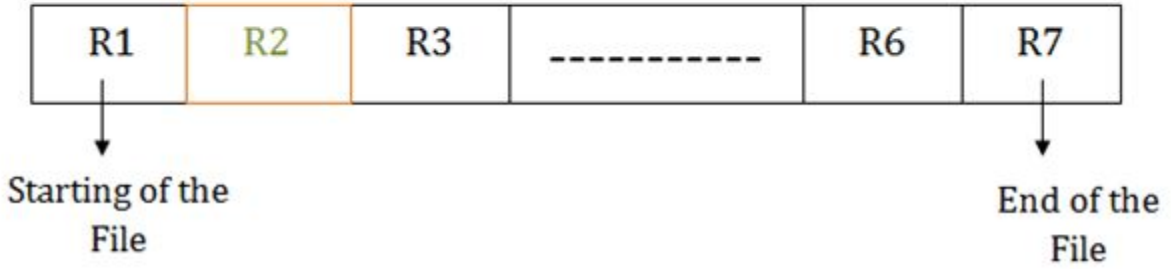
- In this method, the new record is always inserted at the file's end, and then it will sort the sequence in ascending or descending order.
- Sorting of records is based on any primary key or any other key.
- In the case of modification of any record, it will update the record and then sort the file, and lastly the updated record is placed in the right place.



Insertion of the new record:

- Suppose there is a preexisting sorted sequence of four records R1, R3 and so on upto R6 and R7
- Suppose a new record R2 has to be inserted in the sequence, then it will be inserted at the end of the file, and then it will be sorted

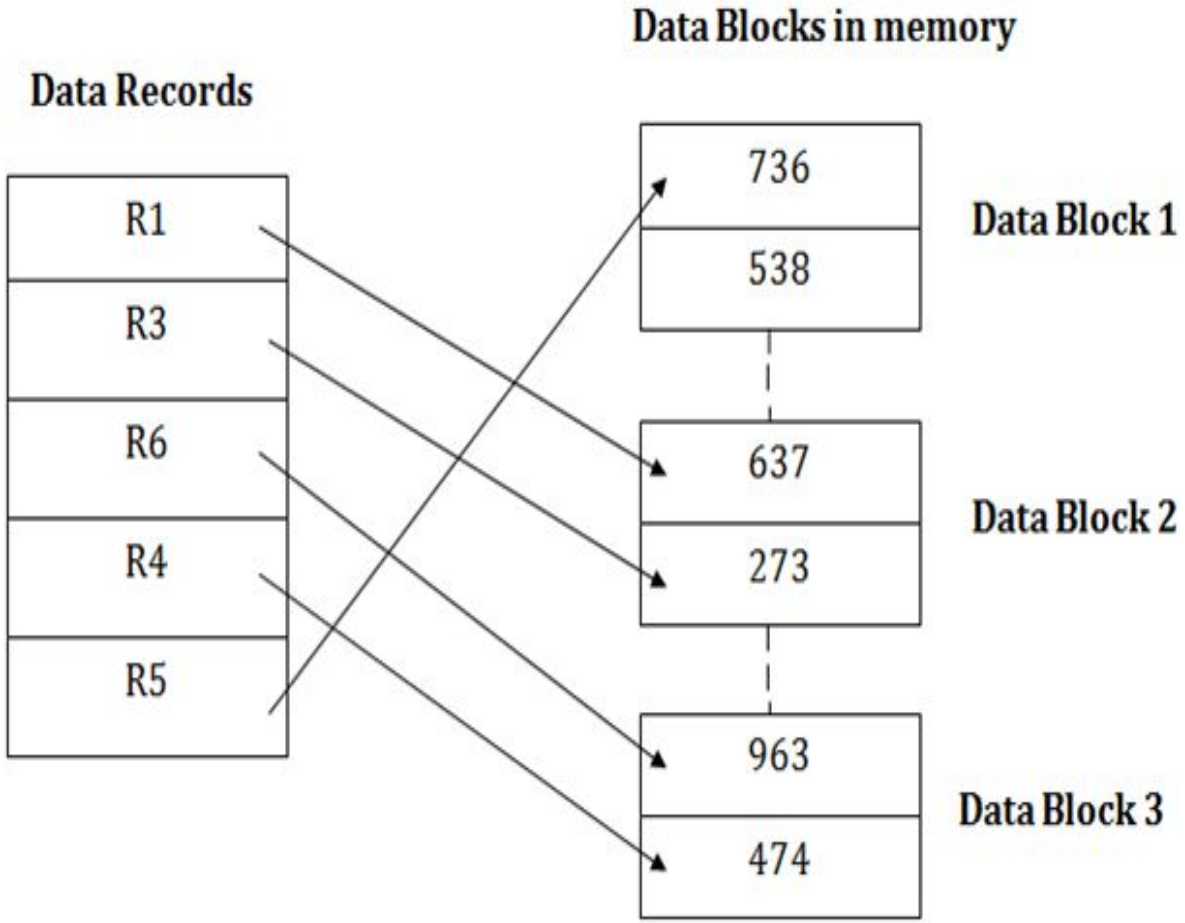




FILE ORGANIZATION

2. Heap File Organization

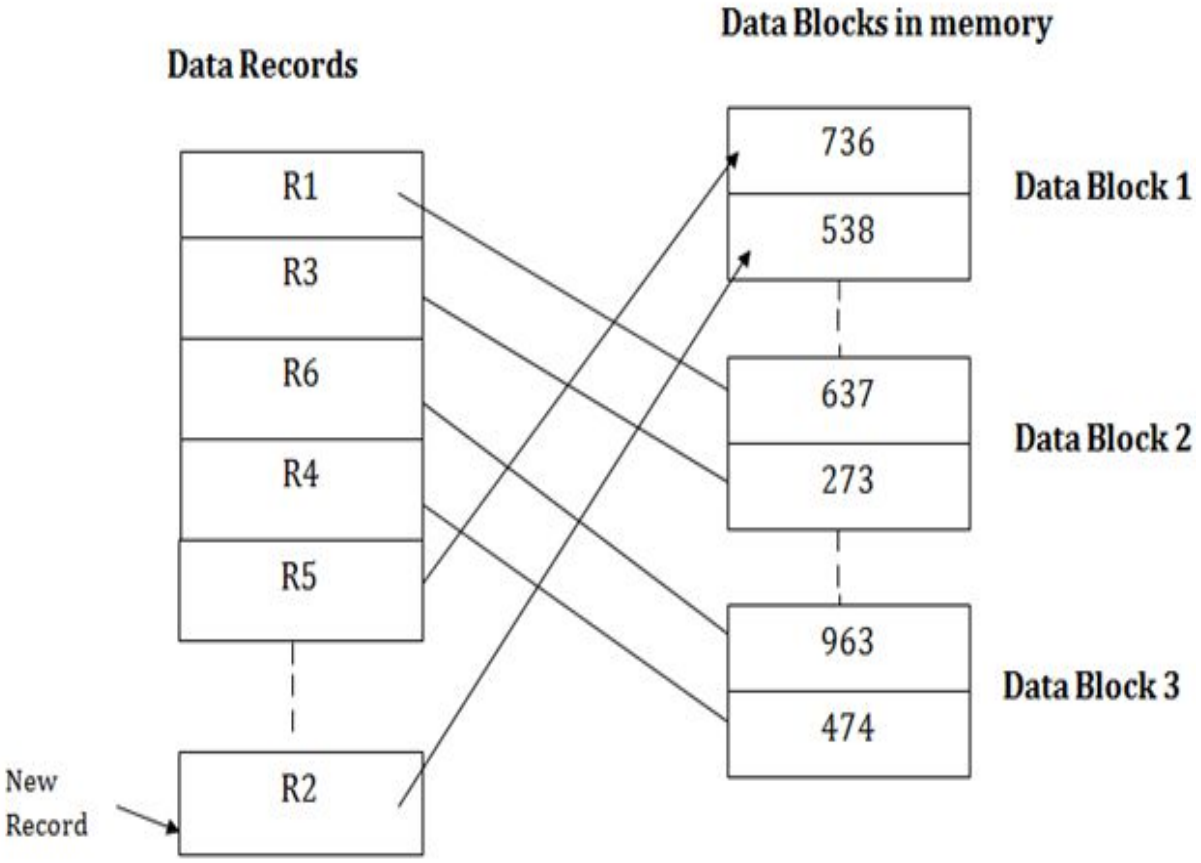
- It is the simplest and most basic type of organization. It works with data blocks. In heap file organization, the records are inserted at the file's end. When the records are inserted, it doesn't require the sorting and ordering of records.
- When the data block is full, the new record is stored in some other block. This new data block need not to be the very next data block, but it can select any data block in the memory to store new records. The heap file is also known as an unordered file.
- In the file, every record has a unique id, and every page in a file is of the same size. It is the DBMS responsibility to store and manage the new records.



FILE ORGANIZATION

Insertion of a new record

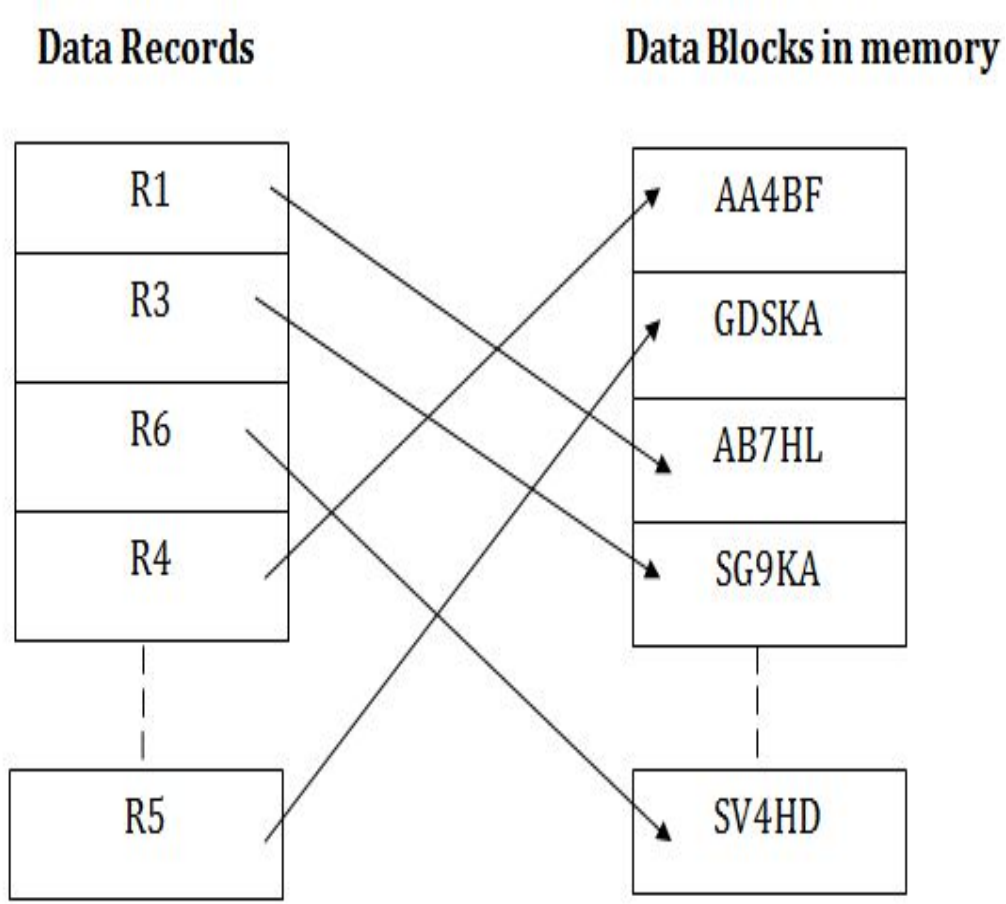
- Suppose we have five records R1, R3, R6, R4 and R5 in a heap and suppose we want to insert a new record R2 in a heap
- If the data block 3 is full then it will be inserted in any of the database selected by the DBMS, let's say data block 1.
- If we want to search, update or delete the data in heap file organization, then we need to traverse the data from staring of the file till we get the requested record.
- If the database is very large then searching, updating or deleting of record will be time-consuming because there is no sorting or ordering of records
- In the heap file organization, we need to check all the data until we



FILE ORGANIZATION

3. Hash File Organization

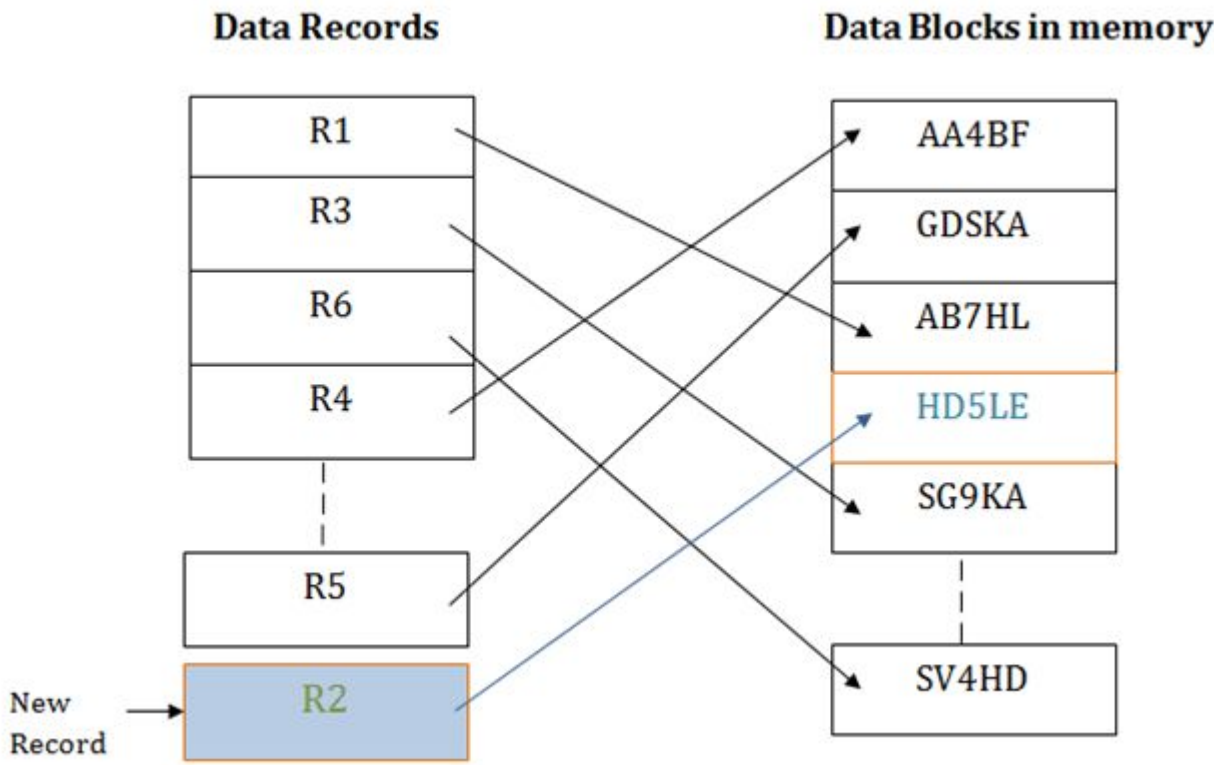
- Hash File Organization uses the computation of hash function on some fields of the records
- The hash function's output determines the location of disk block where the records are to be placed.



FILE ORGANIZATION

Insertion of a new record

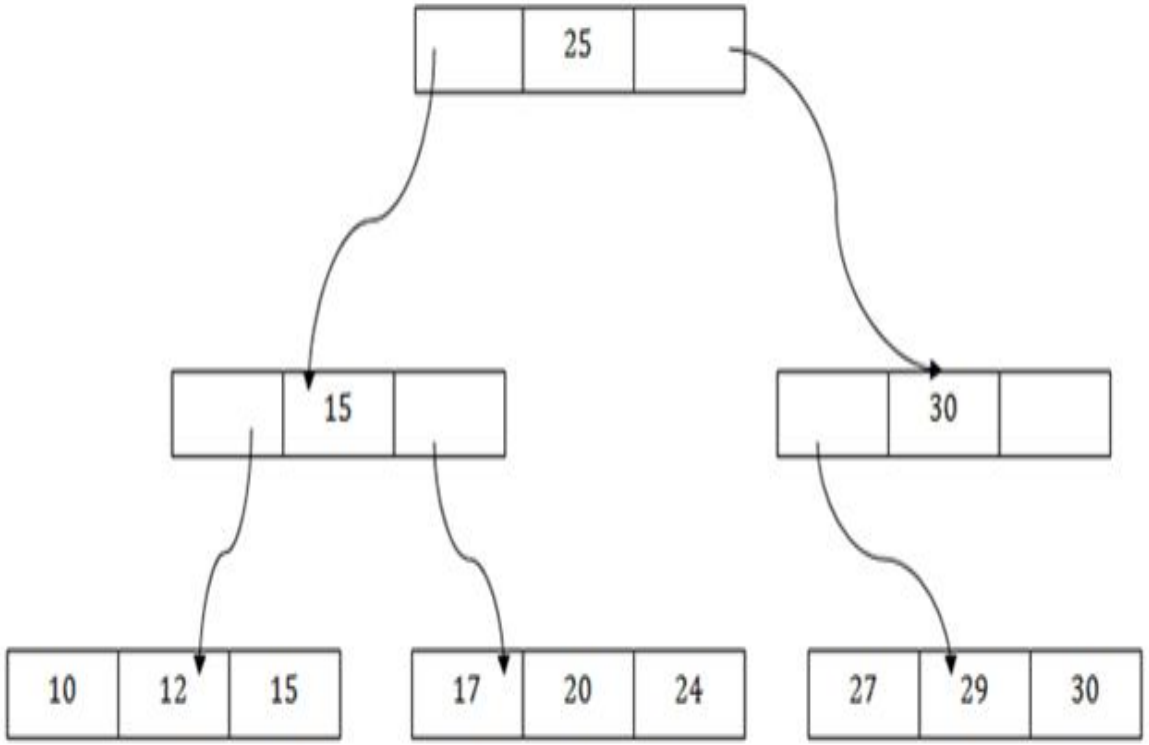
- When a new record has to be inserted, then the address is generated using the hash key and record is directly inserted
- When a record has to be retrieved using the hash key columns, then the address is generated, and the whole record is retrieved using that address
- The same process is applied in the case of delete and update.
- In this method, there is no need for searching and sorting the entire file. In this method, each record will be stored randomly in the memory



FILE ORGANIZATION

4. B+ File Organization

- B+ tree file organization is the advanced method of an indexed sequential access method. It uses a tree-like structure to store records in File.
- It uses the same concept of key-index where the primary key is used to sort the records.
- For each primary key, the value of the index is generated and mapped with the record.
- The B+ tree is similar to a binary search tree (BST), but it can have more than two children.
- In this method, all the records are stored only at the leaf node. Intermediate nodes act as a pointer to the leaf nodes. They do not contain any records.



FILE ORGANIZATION

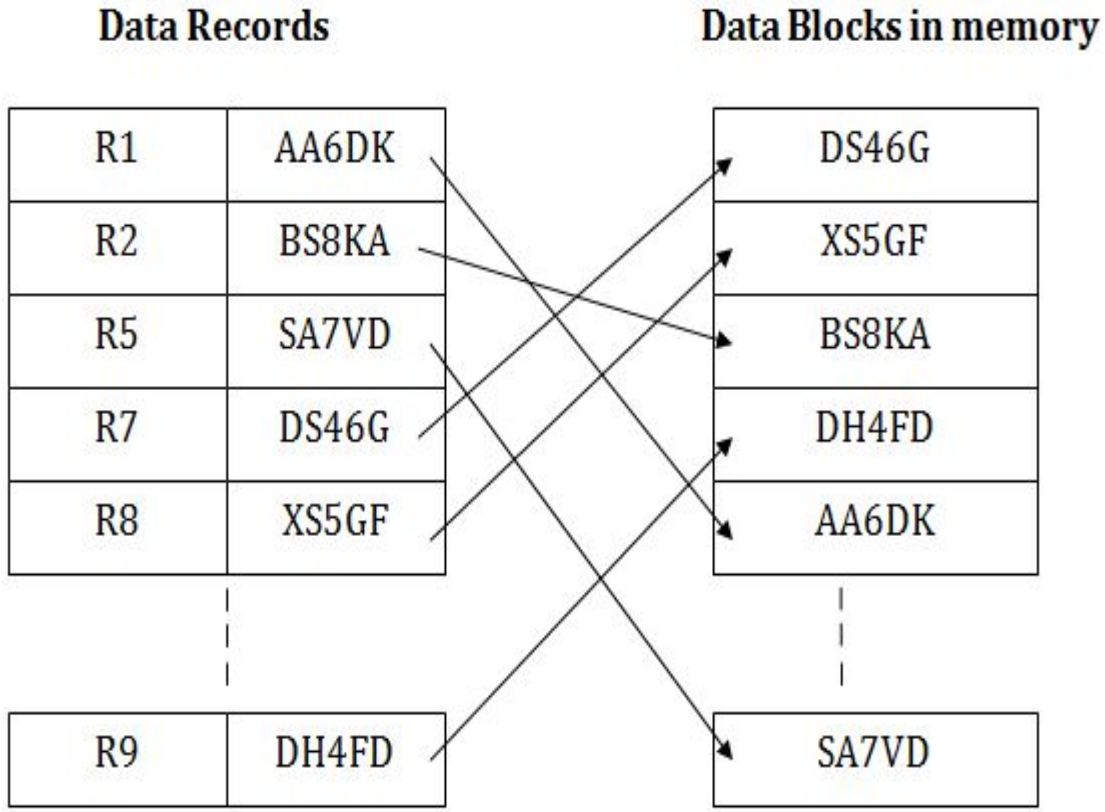
4. B+ File Organization

- The above B+ tree shows that:
  - There is one root node of the tree, i.e., 25.
  - There is an intermediary layer with nodes. They do not store the actual record. They have only pointers to the leaf node.
  - The nodes to the left of the root node contain the prior value of the root and nodes to the right contain next value of the root, i.e., 15 and 30 respectively.
  - There is only one leaf node which has values, i.e., 10, 12, 17, 20, 24, 27 and 29.
  - Searching for any record is easier as all the leaf nodes are balanced.
  - In this method, searching any record can be traversed through the single path and accessed easily

FILE ORGANIZATION

5. Indexed sequential access method (ISAM) File Organization

- ISAM method is an advanced sequential file organization.
- In this method, records are stored in the file using the primary key.
- An index value is generated for each primary key and mapped with the record.
- This index contains the address of the record in the file.
- If any record has to be retrieved based on its index value, then the address of the data block is fetched and the record is retrieved from the memory.



FILE ORGANIZATION

6. Cluster File Organization

- When the two or more records are stored in the same file, it is known as clusters.
- These files will have two or more tables in the same data block
- Key attributes which are used to map these tables together are stored only once
- This method reduces the cost of searching for various records in different files.
- The cluster file organization is used when there is a frequent need for joining the tables with the same condition. These joins will give only a few records from both tables
- In the given example, we are retrieving the record for only particular departments. This method can't be used to retrieve the record for the entire department.
- Data is sorted based on the key with which searching is done. Cluster key is a type of key with which joining of the table is performed.

EMPLOYEE

EMP_ID	EMP_NAME	ADDRESS	DEP_ID
1	John	Delhi	14
2	Robert	Gujarat	12
3	David	Mumbai	15
4	Amelia	Meerut	11
5	Kristen	Noida	14
6	Jackson	Delhi	13
7	Amy	Bihar	10
8	Sonoo	UP	12

DEPARTMENT

DEP_ID	DEP_NAME
10	Math
11	English
12	Java
13	Physics
14	Civil
15	Chemistry

Cluster Key

DEP_ID	DEP_NAME	EMP_ID	EMP_NAME	ADDRESS
10	Math	7	Amy	Bihar
11	English	4	Amelia	Meerut
12	Java	2	Robert	Gujarat
12		8	Sonoo	UP
13	Physics	6	Jackson	Delhi
14	Civil	1	John	Delhi
14		5	Kristen	Noida
15	Chemistry	3	David	Mumbai

FILE ORGANIZATION

Types of Cluster file organization:

- Cluster file organization is of two types:

1. Indexed Clusters:

- In indexed cluster, records are grouped based on the cluster key and stored together.
- The above EMPLOYEE and DEPARTMENT relationship is an example of an indexed cluster
- Here, all the records are grouped based on the cluster key- DEP_ID and all the records are grouped.

2. Hash Clusters:

- It is similar to the indexed cluster
- In hash cluster, we generate the value of the hash key for the cluster key and store the records with the same hash key value.

Data Dictionary Storage

- Data dictionary (also called system catalog) stores metadata: that is, data about data, such as
- Information about relations
  - names of relations
  - names and types of attributes of each relation
  - names and definitions of views
  - integrity constraints
- User and accounting information, including passwords
- Statistical and descriptive data
  - number of tuples in each relation
- Physical file organization information
  - How relation is stored (sequential/hash/…)
  - Physical location of relation
- Information about indices